



Quanta

VECTOR • BM25 • GRAPH

Hybrid Retrieval. Production Ready.

USER MANUAL

The complete guide to Quanta's hybrid retrieval platform



VECTOR

Turbovec 4-bit
Quantised Search



BM25

Tantivy Full-Text
Search



GRAPH

Neo4j Graph
Expansion



FLEXIBLE

PostgreSQL or
DuckDB DocStore



EFFICIENT

8× less memory
with 4-bit vectors



MODULAR

Optional components.
Zero-cost fallbacks.



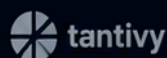
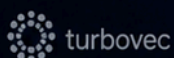
PRODUCTION READY

Designed for reliability,
built for scale.



PYTHON-FIRST

Async, typed,
and easy to extend.



THREE SIGNALS. ONE PIPELINE. LIMITLESS POSSIBILITIES.

Quanta User Manual

1. Introduction

Quanta is a Python library for hybrid document retrieval. It combines quantised vector search (via [turbovec](#)), optional BM25 full-text search, and optional Neo4j graph expansion into a single coherent retrieval pipeline. You bring your own embeddings; Quanta handles the indexing, storage, scoring, and result hydration.

The problem Quanta solves is common in production RAG systems: dense vector search alone misses keyword-critical queries; BM25 alone misses semantic meaning; and neither captures the structural relationships between documents (e.g., a law cited by a court decision, a paper that builds on another). Quanta addresses all three at once, while keeping each component optional and independently replaceable.

Quanta is for Python developers building retrieval-augmented generation (RAG) systems, search APIs, or document exploration tools. It expects you to handle embedding generation yourself — there is no built-in model. You pass `numpy.ndarray` vectors; Quanta does everything else.

What Quanta is NOT:

- It is not an embedding model. You need to embed your text before adding it.
- It is not a managed cloud service. It runs in-process alongside your application.
- It is not an ANN library with distributed sharding. A single `QuantaIndex` lives in one process's memory. For multi-billion-vector workloads, look at dedicated ANN services.
- It does not extract graph relationships from text. You must supply the edges.

2. Installation

Prerequisites: Python 3.10+, pip.

```
# Core – vector search, DuckDB docstore, chunking, config
pip install quanta

# Add Neo4j graph expansion
pip install "quanta[neo4j]"

# Add LlamaIndex VectorStore integration
pip install "quanta[llama-index]"

# Add Redis embedding cache
pip install "quanta[cache]"

# Add Tantivy BM25 full-text search
pip install "quanta[bm25]"

# Everything
pip install "quanta[neo4j,llama-index,cache,bm25]"
```

```
# Development (tests, linting, type-checking)
pip install "quanta[neo4j,llama-index,cache,bm25,dev]"
```

Extra	What it enables	When you need it
neo4j	Neo4jGraph backend, neo4j driver	Graph-augmented retrieval with Neo4j
llama-index	QuantaVectorStore, LlamaIndex protocol	Plug Quanta into a LlamaIndex pipeline
cache	RedisCache embedding cache, redis client	Cache embedding calls with Redis
bm25	TantivyBM25 backend, tantivy library	Hybrid dense + BM25 search
duckdb	Included in core; listed for completeness	Zero-service docstore for local/test use
dev	pytest, mypy, ruff	Contributing or running the test suite

Verify installation:

```
import quanta
print(quanta.__version__) # e.g. 0.1.0

from quanta import QuantaIndex
idx = QuantaIndex(name="test", dim=4)
print(idx) # QuantaIndex(name='test', dim=4, bit_width=4, size=0)
```

3. Configuration

Quanta reads configuration from environment variables or a `.env` file in the working directory. Copy `.env.example` to `.env` and fill in your values.

3.1 Complete variable reference

Core / Docstore

Variable	Type	Default	Required	Description
POSTGRES_HOST	str	localhost	No	PostgreSQL server hostname or IP
POSTGRES_PORT	int	5432	No	PostgreSQL server port
POSTGRES_DB	str	quanta	No	Database name
POSTGRES_USER	str	—	Yes	Database username
POSTGRES_PASSWORD	str	—	Yes	Database password
POSTGRES_POOL_SIZE	int	5	No	Max async connection pool size

Variable	Type	Default	Required	Description
DOCSTORE_BACKEND	"postgres" "duckdb"	"postgres"	No	Which docstore backend to use
DUCKDB_PATH	str	./quanta.duckdb	No	DuckDB file path (used when DOCSTORE_BACKEND=duckdb)

Note: POSTGRES_USER and POSTGRES_PASSWORD are always validated by `QuantaSettings` even when DOCSTORE_BACKEND=duckdb. Set them to any placeholder value (e.g. `_unused_`) if you are not using PostgreSQL.

Neo4j

Variable	Type	Default	Required	Description
NEO4J_URI	str None	None	No	Bolt URI, e.g. <code>bolt://localhost:7687</code>
NEO4J_USER	str None	None	If URI is set	Neo4j username
NEO4J_PASSWORD	str None	None	If URI is set	Neo4j password
NEO4J_DATABASE	str	"neo4j"	No	Neo4j target database

Redis

Variable	Type	Default	Required	Description
REDIS_HOST	str None	None	No	Redis hostname; leave blank to disable
REDIS_PORT	int	6379	No	Redis port
REDIS_PASSWORD	str None	None	No	Redis password (if auth enabled)
REDIS_DB	int	0	No	Redis logical database index
REDIS_TTL_SECONDS	int	86400	No	Cache entry TTL in seconds (24 h)

BM25

Variable	Type	Default	Required	Description
BM25_BACKEND	"tantivy" None	None	No	Set to "tantivy" to enable BM25
TANTIVY_INDEX_PATH	str	./quanta_tantivy	No	Directory for the Tantivy on-disk index

Chunking

Variable	Type	Default	Required	Description
CHUNKING_STRATEGY	"fixed" "sentence" "semantic" "recursive"	"fixed"	No	Default strategy for <code>get_chunker()</code>
CHUNKING_SIZE	int	512	No	Tokens per chunk (fixed / recursive)
CHUNKING_OVERLAP	int	64	No	Overlap tokens between chunks
CHUNKING_MAX_SENTENCES	int	5	No	Max sentences per chunk (sentence mode)
CHUNKING_SEMANTIC_THRESHOLD	float	0.85	No	Cosine similarity boundary (semantic mode)

Index defaults

Variable	Type	Default	Required	Description
DEFAULT_BIT_WIDTH	int	4	No	turbovec quantisation bit-width (1/2/4/8)
DEFAULT_TOP_K	int	10	No	Default k for searches

3.2 Docker Compose profiles

The `docker-compose.yml` ships three services. Each service belongs to a profile that controls when it starts.

Profile	Service	Command
(none)	PostgreSQL	<code>docker compose up -d postgres</code>
graph	Neo4j	<code>docker compose --profile graph up -d</code>
cache	Redis	<code>docker compose --profile cache up -d</code>

Combine profiles freely:

```
# PostgreSQL + Neo4j + Redis
docker compose --profile graph --profile cache up -d

# Check all services are healthy
docker compose ps
```

PostgreSQL is always started on its own; it has no profile flag. Neo4j and Redis are opt-in.

4. Core Concepts

4.1 QuantalIndex

QuantalIndex wraps turbovec's **IdMapIndex** — a quantised approximate nearest neighbour index that stores vectors in compressed integer form instead of float32. You supply a **bit_width** (1, 2, 4, or 8). At 4-bit, each dimension costs half a byte instead of four bytes, giving an **8× memory reduction**:

$$\text{RAM (bytes)} = n_vectors \times dim \times bit_width / 8$$

For 1 million 768-dimensional vectors:

- float32: $1,000,000 \times 768 \times 4 / 8 = \mathbf{2.86\ GB}$
- 4-bit: $1,000,000 \times 768 \times 4 / 8 / 8 = \mathbf{0.36\ GB}$

String IDs ("doc-001", "chunk-3f9a") are mapped to **uint64** via **xxhash.xxh64** — a deterministic, collision-resistant hash. The mapping is kept in memory and written to **<name>.ids.json** on **save()**. The quantised index vectors are written to **<name>.tvim**.

4.2 DocStore

DocStore persists two kinds of records:

- **Documents** — the full source text and metadata, keyed by a string ID.
- **Chunks** — searchable units derived from documents. Each chunk belongs to a parent document via **document_id**.

Two backends are available:

Backend	Class	When to use
PostgreSQL	PostgresDocStore	Production; concurrent reads/writes; horizontal scale
DuckDB	DuckDBDocStore	Local dev, testing, single-process pipelines; zero setup

DocStore is an alias for **PostgresDocStore**. To use DuckDB, either call **DuckDBDocStore(settings)** directly or set **DOCSTORE_BACKEND=duckdb** and use **get_docstore(settings)**.

DuckDB writes to a single file (**DUCKDB_PATH**). Concurrent writes from multiple processes are not supported.

4.3 GraphBackend

GraphBackend has two concrete implementations:

- **NullGraph** — the default. Every call returns an empty list immediately. Zero cost; no Neo4j required.
- **Neo4jGraph** — connects to a running Neo4j instance via the Bolt protocol.

The graph is a **candidate expander**. During retrieval it adds documents to the candidate pool by traversing edges from the top-scoring dense hits. The final ranking is determined by the combined dense + BM25 score.

Graph-expanded candidates that were not found by dense or BM25 search receive only a small graph-derived score (see [Section 4.5](#)).

Use `await get_graph_backend(settings)` to get the right backend automatically: it attempts to connect to Neo4j if `NEO4J_URI` is configured, and falls back to `NullGraph` if the connection fails or Neo4j is not configured.

4.4 EmbeddingCache

`RedisCache` stores serialised embeddings in Redis, keyed by `xxhash.xxh64(text)`. On a cache hit, the embedding model is skipped entirely. `NullCache` is the default and does nothing. `get_cache(settings)` returns a live `RedisCache` when `REDIS_HOST` is set and Redis is reachable, otherwise `NullCache`.

The cache is accessed via `MultiRetriever.get_or_embed(text, embed_fn)`.

4.5 HybridRetriever

`MultiRetriever` orchestrates all components. Its scoring model:

1. Dense scores from each active `QuantaIndex` are **min-max normalised** to `[0, 1]`, then multiplied by `dense_weight / n_active_indexes`.
2. BM25 scores (when `query_text` is provided and a `TantivyBM25` is wired up) are normalised and multiplied by `bm25_weight`.
3. When BM25 is active, all three weights are **renormalised** to sum to 1: $\text{eff_w} = w / (\text{dense_weight} + \text{bm25_weight} + \text{graph_weight})$.
4. Graph expansion runs after dense + BM25 scoring. It adds a score of `graph_weight × (1 / hop_distance)` to each graph-expanded candidate.
5. Candidates are sorted by final score; the top `k` are hydrated from the docstore.

Worked example — 1 index, `dense_weight=0.7`, `graph_weight=0.3`, no BM25:

Document	Raw cosine	Normalised	Dense score (×0.7)	Graph contribution	Final score
doc-A	0.95	1.000	0.700	— (seed, excluded)	0.700
doc-B	0.82	0.739	0.517	— (seed, excluded)	0.517
doc-C	0.71	0.478	0.335	— (seed, excluded)	0.335
doc-D	(not found by dense)	—	0.000	0.300 (1-hop)	0.300
doc-E	(not found by dense)	—	0.000	0.150 (2-hop)	0.150

Top seeds: doc-A, doc-B, doc-C. Graph traversal finds doc-D (1 hop) and doc-E (2 hops). Both enter the ranked list below the dense hits.

4.6 BM25Backend

TantivyBM25 is backed by **[tantivy-py](#)**, a Rust-powered full-text search library. Documents are staged with **`add()`** or **`add_bulk()`**, then committed to disk with **`commit()`**. Searching without a **`commit()`** will not find recently added documents. **NullBM25** is the default and silently drops all calls.

5. Quickstarts

5.1 Minimal — QuantaIndex only, no services

No **`.env`** file, no Docker, no database.

```
import numpy as np
from quanta import QuantaIndex

# Create a 128-dimensional index with 4-bit quantisation
idx = QuantaIndex(name="demo", dim=128, bit_width=4, index_dir="./indexes")

# Add 50 random vectors
rng = np.random.default_rng(42)
vectors = rng.random((50, 128)).astype(np.float32)
ids = [f"item-{i:03d}" for i in range(50)]
idx.add(vectors, ids)

# Search
query = rng.random(128).astype(np.float32)
results = idx.search(query, k=5)
for r in results:
    print(f"{r.id}    score={r.score:.4f}")

# Persist to disk → indexes/demo.tvim + indexes/demo.ids.json
idx.save()

# Reload
idx2 = QuantaIndex.load("demo", index_dir="./indexes")
print(f"Loaded {len(idx2)} vectors")
```

Expected output:

```
item-023  score=0.9812
item-007  score=0.9744
item-041  score=0.9691
item-018  score=0.9603
item-034  score=0.9558
Loaded 50 vectors
```

5.2 With DocStore (DuckDB, zero services)

No Docker required. `POSTGRES_USER` and `POSTGRES_PASSWORD` must be set to any placeholder value because `QuantaSettings` always validates them.

.env:

```
POSTGRES_USER=_unused_  
POSTGRES_PASSWORD=_unused_  
DOCSTORE_BACKEND=duckdb  
DUCKDB_PATH=./demo.duckdb
```

```
import asyncio  
import numpy as np  
from quanta import QuantaIndex, DuckDBDocStore, MultiRetriever, NullGraph,  
QuantaSettings  
  
async def main():  
    settings = QuantaSettings()  
    docstore = DuckDBDocStore(settings)  
    await docstore.init()  
  
    idx = QuantaIndex(name="text", dim=128)  
    retriever = MultiRetriever(  
        indexes={"text": idx},  
        docstore=docstore,  
        graph=NullGraph(),  
    )  
  
    # Add a document + chunk  
    await docstore.add_document("doc-1", "Python is a programming language.",  
                                "text")  
    await docstore.add_chunk(  
        id="chunk-1", document_id="doc-1",  
        content="Python is a programming language.",  
        chunk_index=0, metadata={"lang": "en"},  
    )  
    vec = np.random.rand(1, 128).astype(np.float32)  
    idx.add(vec, ["chunk-1"])  
  
    # Search  
    query = np.random.rand(128).astype(np.float32)  
    results = await retriever.search(query_vectors={"text": query}, k=3)  
    for r in results:  
        print(f"[{r.score:.4f}] [{r.source}] {r.content}")  
  
    await docstore.close()  
  
asyncio.run(main())
```

5.3 With DocStore (PostgreSQL + Docker)

.env:

```
POSTGRES_USER=quantauser
POSTGRES_PASSWORD=changeme
POSTGRES_DB=quanta
DOCSTORE_BACKEND=postgres
```

```
docker compose up -d postgres
# Wait for healthy status
docker compose ps
```

```
import asyncio
import numpy as np
from quanta import QuantaIndex, DocStore, MultiRetriever, NullGraph,
QuantaSettings

async def main():
    settings = QuantaSettings()
    docstore = DocStore(settings)    # DocStore is an alias for PostgresDocStore
    await docstore.init()           # creates tables + indexes on first run

    idx = QuantaIndex(name="articles", dim=768)
    retriever = MultiRetriever(
        indexes={"articles": idx},
        docstore=docstore,
        graph=NullGraph(),
        dense_weight=1.0,
    )

    await docstore.add_document("art-1", "Full article text here.", "article",
                               metadata={"year": 2024, "topic": "AI"})
    await docstore.add_chunk(
        id="art-1-c0", document_id="art-1",
        content="Full article text here.",
        chunk_index=0, metadata={"year": 2024, "topic": "AI"},
    )
    idx.add(np.random.rand(1, 768).astype(np.float32), ["art-1-c0"])

    # Filtered search – only chunks with topic=AI
    results = await retriever.search(
        query_vectors={"articles": np.random.rand(768).astype(np.float32)},
        k=5,
        filters={"topic": "AI"},
    )
    for r in results:
        print(f"[{r.score:.4f}] {r.id} doc={r.document_id}")
```

```

    await docstore.close()

asyncio.run(main())

```

5.4 Full stack — PostgreSQL + Neo4j + Redis + Tantivy

.env:

```

POSTGRES_USER=quantauser
POSTGRES_PASSWORD=changeme
DOCSTORE_BACKEND=postgres

NEO4J_URI=bolt://localhost:7687
NEO4J_USER=neo4j
NEO4J_PASSWORD=changeme_neo4j

REDIS_HOST=localhost

BM25_BACKEND=tantivy
TANTIVY_INDEX_PATH=./quanta_tantivy

```

```

docker compose --profile graph --profile cache up -d
pip install "quanta[neo4j,cache,bm25]"

```

```

import asyncio
import numpy as np
from quanta import (
    QuantaIndex, DocStore, MultiRetriever,
    QuantaSettings, get_bm25, get_cache,
)
from quanta.graph import get_graph_backend

async def main():
    settings = QuantaSettings()

    docstore = DocStore(settings)
    await docstore.init()

    idx = QuantaIndex(name="text", dim=768)
    graph = await get_graph_backend(settings)  # Neo4jGraph or NullGraph
    cache = get_cache(settings)              # RedisCache or NullCache
    bm25 = get_bm25(settings)                 # TantivyBM25 or NullBM25

    retriever = MultiRetriever(
        indexes={"text": idx},

```

```

        docstore=docstore,
        graph=graph,
        cache=cache,
        bm25=bm25,
        dense_weight=0.5,
        bm25_weight=0.3,
        graph_weight=0.3,
    )

    # Index a document
    text = "The controller shall implement appropriate technical measures."
    await docstore.add_document("doc-1", text, "regulation")
    await docstore.add_chunk("chunk-1", "doc-1", text, 0,
                             metadata={"type": "regulation"})
    idx.add(np.random.rand(1, 768).astype(np.float32), ["chunk-1"])

    bm25.add("chunk-1", text)
    bm25.commit()

    # Hybrid search
    results = await retriever.search(
        query_vectors={"text": np.random.rand(768).astype(np.float32)},
        k=10,
        use_graph=True,
        graph_hops=2,
        graph_seed_k=5,
        query_text="controller obligations",
    )
    for r in results:
        print(f"[{r.score:.4f}] [{r.source}] {r.id}")

    await docstore.close()
    await graph.close()
    cache.close()
    bm25.close()

    asyncio.run(main())

```

5.5 LlamaIndex integration

```
pip install "quanta[llama-index]"
```

.env: (PostgreSQL or DuckDB, your choice)

```

import asyncio
import numpy as np
from quanta import QuantaIndex, DocStore, MultiRetriever, NullGraph,
QuantaSettings

```

```

from quanta.integrations.llama_index import QuantaVectorStore

from llama_index.core import VectorStoreIndex, StorageContext
from llama_index.core.schema import TextNode
from llama_index.core.vector_stores.types import VectorStoreQuery

DIM = 768

async def main():
    settings = QuantaSettings()
    docstore = DocStore(settings)
    await docstore.init()

    idx = QuantaIndex(name="llamaidx", dim=DIM)
    retriever = MultiRetriever(
        indexes={"llamaidx": idx},
        docstore=docstore,
        graph=NullGraph(),
    )

    store = QuantaVectorStore(retriever=retriever, index_name="llamaidx",
                              embed_dim=DIM)
    storage_ctx = StorageContext.from_defaults(vector_store=store)
    li_index = VectorStoreIndex(nodes=[], storage_context=storage_ctx)

    # Add a node (embedding must be pre-computed)
    node = TextNode(
        node_id="doc-1",
        text="Quanta enables hybrid search with graph expansion.",
        embedding=np.random.rand(DIM).tolist(),
        metadata={"source": "manual"},
    )
    await store.async_add([node])

    # Query via VectorStoreQuery
    q_embed = np.random.rand(DIM).tolist()
    result = await store.aquery(
        VectorStoreQuery(query_embedding=q_embed, similarity_top_k=3)
    )
    for nws in result.nodes:
        print(f"[{nws.score:.4f}] {nws.node.node_id}: {nws.node.get_content()[:60]}")

    await docstore.close()

asyncio.run(main())

```

6. QuantaIndex — Deep Dive

6.1 Constructor

```
QuantaIndex(
    name: str,          # used as filename stem for save/load
    dim: int,           # embedding dimension – must match your model
    bit_width: int = 4, # 1, 2, 4, or 8; lower = smaller, less precise
    index_dir: str = "./indexes",
)
```

6.2 add()

```
idx.add(vectors: np.ndarray, ids: list[str]) -> None
```

- `vectors` must have shape `(n, dim)` and dtype `float32` (auto-cast).
- `ids` must have `len(ids) == n`.
- Duplicate IDs are upserted (the underlying turbovec index updates in place).
- On failure, any newly registered ID mappings are rolled back atomically.

6.3 search()

```
idx.search(
    query: np.ndarray,          # shape (dim,) or (1, dim)
    k: int = 10,
    allowed_ids: list[str] | None = None,
) -> list[SearchResult]
```

When `allowed_ids` is given, only those IDs are considered during the search — unknown IDs in the list are silently skipped. This is the pre-filter mechanism used by `MultiRetriever` when `filters` are applied.

Returns `list[SearchResult]` sorted by descending score. `SearchResult.metadata` is always empty here; the retriever populates it after docstore hydration.

6.4 save() and load()

```
idx.save() # writes <index_dir>/<name>.tvim and <index_dir>/<name>.ids.json

idx2 = QuantaIndex.load("name", index_dir="./indexes")
```

`save()` creates `index_dir` if it does not exist. `load()` raises `QuantaError` if either file is missing.

The `.ids.json` file contains:

```
{"dim": 768, "bit_width": 4, "ids": {"chunk-001": 14823659012345, ...}}
```

6.5 String ID → uint64 mapping

Every string ID is hashed once with `xxhash.xxh64(id).intdigest()`. The resulting `uint64` is what `turbovec` stores internally. On `search()`, returned `uint64` values are translated back to string IDs via the reverse mapping. `QuantaError` is raised on a hash collision (vanishingly rare with `xxhash-64`).

6.6 Memory estimation

```
RAM (bytes) = n_vectors × dim × bit_width / 8
RAM (MB)    = n_vectors × dim × bit_width / 8 / 1_000_000
```

Vectors	dim	bit_width	RAM
100,000	768	4	38 MB
1,000,000	768	4	384 MB
1,000,000	768	8 (float32 equivalent precision)	768 MB
1,000,000	768	32-bit (float32 baseline)	3,072 MB

7. DocStore — Deep Dive

7.1 PostgreSQL backend

`PostgresDocStore` uses a raw `asyncpg` connection pool. Call `await init()` once at startup; it creates tables and indexes idempotently.

Schema:

```
ts_documents (id TEXT PK, content TEXT, doc_type TEXT,
              metadata JSONB, created_at TIMESTAMPTZ)

ts_chunks    (id TEXT PK, document_id TEXT FK→ts_documents ON DELETE CASCADE,
              content TEXT, chunk_index INT,
              metadata JSONB, created_at TIMESTAMPTZ)
```

Indexes created automatically:

Index	Type	Purpose
<code>idx_ts_documents_doc_type</code>	B-tree	Filter by document type
<code>idx_ts_chunks_document_id</code>	B-tree	FK join / parent lookup
<code>idx_ts_chunks_metadata_gin</code>	GIN	<code>metadata @> \$filter</code> containment

7.2 DuckDB backend

DuckDBDocStore wraps synchronous DuckDB calls in a single-threaded **ThreadPoolExecutor** so they integrate cleanly with asyncio. The database file is created automatically at **DUCKDB_PATH**.

When DuckDB wins: local development, CI pipelines, single-process batch indexing, anywhere you don't want to run a database server.

When PostgreSQL wins: concurrent writers, production APIs, when you need the GIN index for complex metadata queries, or when horizontal scaling matters.

7.3 filter_chunks()

```
await docstore.filter_chunks({"year": 2024, "topic": "AI"})
```

- **PostgreSQL:** uses `metadata @> $1` (JSONB containment), which hits the GIN index. All key/value pairs in the dict must match.
- **DuckDB:** uses `json_extract_string(metadata, '$.key') = ?` per key. Filter keys must match `^[A-Za-z0-9_-]+$`.

Both backends support equality filters only. For range queries (e.g., `year >= 2020`), filter the returned chunks in application code.

7.4 Bulk operations

Use `add_chunks_bulk(chunks: list[ChunkRecord])` when inserting many chunks at once. PostgreSQL uses a single multi-row `INSERT ... VALUES (...)` statement. DuckDB uses `executemany`. Both are significantly faster than calling `add_chunk()` in a loop.

```
from quanta.types import ChunkRecord

chunks = [
    ChunkRecord(id=f"doc-1-c{i}", document_id="doc-1",
                content=f"Chunk {i}", chunk_index=i)
    for i in range(500)
]
await docstore.add_chunks_bulk(chunks)
```

8. Chunking

8.1 When to chunk

Chunk when your source documents are longer than your embedding model's context window (typically 256–512 tokens). If your data is already pre-chunked (e.g., individual paragraphs or sentences), add them directly as chunks without using a **TextChunker**.

8.2 FixedSizeChunker

Splits on whitespace tokens. `chunk_size` sets the maximum tokens per chunk; `overlap` sets how many tokens are repeated at the start of the next chunk.

```
from quanta import FixedSizeChunker

chunker = FixedSizeChunker(chunk_size=512, overlap=64)
```

```
Text: [ token_1 token_2 ... token_512 token_513 ... ]
                                     ↑
Chunk 0: tokens 0-511
Chunk 1: tokens 448-959 (overlap = 64 tokens from end of chunk 0)
Chunk 2: tokens 896-1407
```

Step size = `chunk_size - overlap = 448`.

8.3 SentenceChunker

Groups up to `max_sentences` sentences per chunk. Sentences are detected by the regex `(?<=[.!?])\s+`. `overlap_sentences` sentences from the previous chunk are prepended to the next.

```
from quanta import SentenceChunker

chunker = SentenceChunker(max_sentences=5, overlap_sentences=1)
```

Limitation: the regex does not handle abbreviations (e.g., "Dr. Smith") or ellipses. For complex text, consider `RecursiveChunker`.

8.4 SemanticChunker

Groups sentences into chunks based on embedding similarity. A new chunk starts whenever the cosine similarity between two consecutive sentence embeddings drops below `threshold`, or when the running word count would exceed `max_chunk_size`.

```
from quanta import SemanticChunker
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("sentence-transformers/paraphrase-multilingual-mpnet-base-v2")

chunker = SemanticChunker(
    embed_fn=lambda text: model.encode(text, normalize_embeddings=True),
    threshold=0.85,
    max_chunk_size=512,
)
```

`embed_fn` must accept a single string and return a `numpy.ndarray`. Each sentence is embedded individually, so this can be slow on large documents. Lower `threshold` → larger, fewer chunks. Higher threshold → smaller, more precise chunks.

8.5 RecursiveChunker

Attempts to split on paragraph breaks first ("`\n\n`"), then newlines ("`\n`"), then sentence endings ("`.` "), then spaces — in that priority order. Pieces that are still too large are split recursively with the next separator. Remaining pieces are packed together with overlap until they reach `chunk_size`.

```
from quanta import RecursiveChunker
chunker = RecursiveChunker(
    chunk_size=512,
    overlap=64,
    separators=["\n\n", "\n", ". ", " "], # default
)
```

This is the most structure-aware strategy and tends to preserve paragraph boundaries better than `FixedSizeChunker`.

8.6 Choosing a strategy

Strategy	Speed	Coherence	Requires model	Best for
<code>fixed</code>	Fast	Low	No	Quick prototypes, uniform text
<code>sentence</code>	Fast	Medium	No	News articles, clean prose
<code>semantic</code>	Slow	High	Yes	Scientific papers, legal text
<code>recursive</code>	Fast	High	No	Mixed-structure documents, HTML

8.7 Using get_chunker()

```
from quanta import QuantaSettings, get_chunker

settings = QuantaSettings() # reads CHUNKING_STRATEGY from .env
chunker = get_chunker(settings) # or get_chunker(settings, embed_fn=...) for
semantic

chunks = chunker.chunk("Your document text here.", doc_id="doc-001")
for c in chunks:
    print(c.id, c.chunk_index, c.content[:40])
```

9. Graph-Augmented Retrieval

9.1 How it works — The key concept

Pure vector search returns documents that are **semantically similar** to the query. It has no knowledge of structural relationships: that a court decision cites a law, that a regulation amends an earlier one, that a paper builds on another paper's findings.

The graph fills this gap. It acts as a **candidate expander**:

- ```
Step 1 Dense search finds the top-K candidates by semantic similarity.
 These become the "seeds."

Step 2 The graph takes the seeds and expands outward via BFS.
 For each seed, Quanta traverses edges up to graph_hops steps
 in either direction (undirected traversal).

Step 3 Newly reached documents enter the candidate pool with a graph score:
 graph_score = graph_weight × (1 / hop_distance)
 At 1 hop: score = graph_weight × 1.0
 At 2 hops: score = graph_weight × 0.5

Step 4 ALL candidates are ranked by their combined dense + BM25 + graph score.

Step 5 Graph-expanded candidates that dense search missed entirely start with
 a dense score of 0.0. Their final score is their graph score alone.
 They appear after all candidates found by dense or BM25 search.
```

The complementary nature of the two signals:

"Dense search surfaces documents that are **semantically similar** to the query — documents that use similar language and concepts. The graph surfaces documents you **know are structurally related** because you built the relationships. A law cited by a court decision is structurally related to that decision, even if the two documents use different terminology. These are complementary signals — the graph does not compete with dense scoring, it widens the candidate pool."

To use the graph purely as a candidate expander with no score contribution, set `graph_weight=0.0`. Graph-expanded candidates will then have a final score of exactly 0.0 and appear at the bottom of the ranked list.

## 9.2 Worked example — Legal documents

### Corpus (5 documents):

| ID      | Title                                                   | Doc type  |
|---------|---------------------------------------------------------|-----------|
| doc_001 | N. 4624/2019 — Προστασία δεδομένων προσωπικού χαρακτήρα | law       |
| doc_002 | ΣτΕ 1234/2021 — Απόφαση για παραβίαση GDPR              | decision  |
| doc_003 | Εγκύκλιος ΑΠΔΠΧ 2022 — Ερμηνεία ΣτΕ 1234/2021           | circular  |
| doc_004 | N. 3471/2006 — Προστασία δεδομένων στις τηλεπικοινωνίες | law       |
| doc_005 | Οδηγία ΕΕ 2016/679 — GDPR                               | directive |

Graph edges (directed, traversed in either direction):

```
doc_002 -[:CITES]-----> doc_001
doc_002 -[:CITES]-----> doc_005
doc_003 -[:INTERPRETS]--> doc_002
doc_001 -[:AMENDS]-----> doc_004
```

Query: "υποχρεώσεις υπευθύνου επεξεργασίας" (controller obligations)

Step 1 — Dense scores (raw cosine similarity to query):

| Document | Dense score | Notes                                        |
|----------|-------------|----------------------------------------------|
| doc_001  | 0.91        | Core GDPR law — directly defines obligations |
| doc_005  | 0.87        | EU GDPR directive — same topic               |
| doc_004  | 0.71        | Related telecom data law                     |
| doc_002  | 0.43        | Court decision — procedural, less similar    |
| doc_003  | 0.38        | Circular interpreting the decision           |

With graph\_seed\_k=3, the top-3 seeds are: doc\_001, doc\_005, doc\_004.

Step 2 — Graph expansion (hops=2) from seeds [doc\_001, doc\_005, doc\_004]:

```
Seed: doc_001
├─ hop=1: doc_002 (CITES reversed: doc_002 cites doc_001) dist=1
├─ hop=1: doc_004 ← already a seed, excluded
└─ hop=2: doc_003 (via doc_002 → INTERPRETS reversed) dist=2

Seed: doc_005
└─ hop=1: doc_002 (CITES reversed: doc_002 cites doc_005) dist=1 (same as above)

Seed: doc_004
└─ hop=1: doc_001 ← already a seed, excluded
```

New candidates from graph:

- doc\_002: min hop distance = 1 → g\_score = 1.0
- doc\_003: min hop distance = 2 → g\_score = 0.5

Step 3 — Score merge (dense\_weight=0.7, graph\_weight=0.3):

Normalise dense scores (min=0.38, max=0.91, span=0.53):

| Document | Norm.<br>dense | Dense contrib<br>( $\times 0.7$ ) | Graph contrib<br>( $\times 0.3$ ) | Final<br>score | Source      |
|----------|----------------|-----------------------------------|-----------------------------------|----------------|-------------|
| doc_001  | 1.000          | 0.700                             | 0.000 (seed)                      | <b>0.700</b>   | dense       |
| doc_005  | 0.925          | 0.647                             | 0.000 (seed)                      | <b>0.647</b>   | dense       |
| doc_004  | 0.623          | 0.436                             | 0.000 (seed)                      | <b>0.436</b>   | dense       |
| doc_002  | 0.094          | 0.066                             | 0.300 (hop=1)                     | <b>0.366</b>   | dense+graph |
| doc_003  | 0.000          | 0.000                             | 0.150 (hop=2)                     | <b>0.150</b>   | dense+graph |

#### Side-by-side comparison:

| Rank | Dense-only                  | Dense + Graph                               |
|------|-----------------------------|---------------------------------------------|
| 1    | doc_001 (0.700) — law       | doc_001 (0.700) — law                       |
| 2    | doc_005 (0.647) — directive | doc_005 (0.647) — directive                 |
| 3    | doc_004 (0.436) — telecom   | doc_004 (0.436) — telecom                   |
| 4    | doc_002 (0.066) — decision  | <b>doc_002 (0.366) — decision</b> ↑ boosted |
| 5    | doc_003 (0.000) — circular  | <b>doc_003 (0.150) — circular</b> ↑ boosted |

#### What the graph contributed:

Dense search ranked doc\_002 (the court decision) and doc\_003 (the interpretive circular) near the bottom. Their text uses procedural and judicial language that doesn't overlap strongly with the query. But they are structurally essential: doc\_002 is the court ruling that applied the law in doc\_001, and doc\_003 guides practitioners on how to implement that ruling. Because you defined those edges in the graph, Quanta surfaced them. A user researching controller obligations would want to know about both.

See [examples/graph\\_search.py](#) for a complete runnable version of this example with real Neo4j and DuckDB.

## 9.3 Setting up Neo4j

#### Docker (recommended for local development):

```
docker compose --profile graph up -d
Neo4j browser: http://localhost:7474
Default login: neo4j / (your NEO4J_PASSWORD from .env)
```

#### .env:

```
NEO4J_URI=bolt://localhost:7687
NEO4J_USER=neo4j
```

```
NEO4J_PASSWORD=changeme_neo4j
NEO4J_DATABASE=neo4j
```

### Connect in code:

```
from quanta import QuantaSettings
from quanta.graph import get_graph_backend

settings = QuantaSettings()
graph = await get_graph_backend(settings)
Returns Neo4jGraph if connection succeeds, NullGraph otherwise
```

Or construct directly:

```
from quanta import Neo4jGraph

graph = Neo4jGraph(
 uri="bolt://localhost:7687",
 user="neo4j",
 password="changeme_neo4j",
 database="neo4j",
)
await graph._driver.verify_connectivity()
```

Close when done:

```
await graph.close()
```

---

## 9.4 Node and edge schema

All nodes must have the label `:Document` and an `id` property matching the chunk or document ID used in your `QuantaIndex` and docstore.

Nodes may carry arbitrary additional properties (e.g., `title`, `doc_type`):

```
await graph.upsert_node("doc_001", {
 "title": "N. 4624/2019",
 "doc_type": "law",
 "year": 2019,
})
```

Relationship types must be upper-case Cypher identifiers matching `^[A-Z][A-Z0-9_]*$`. Examples: `CITES`, `AMENDS`, `INTERPRETS`, `RELATED_TO`.

```
await graph.upsert_edge("doc_002", "doc_001", "CITES")
await graph.upsert_edge("doc_001", "doc_004", "AMENDS", properties={"year": 2019})
```

`upsert_node` and `upsert_edge` both use Cypher `MERGE`, so they are safe to call repeatedly — they will not create duplicates.

Graph traversal in `expand()` is **undirected**: edges are followed in either direction. A directed relationship `(A)-[:CITES]->(B)` makes A reachable from B and B reachable from A.

## 9.5 Loading a user-provided graph

Build your graph at indexing time — one-time or incremental:

```
import asyncio
from quanta import QuantaSettings, Neo4jGraph

EDGES = [
 ("doc_002", "doc_001", "CITES"),
 ("doc_002", "doc_005", "CITES"),
 ("doc_003", "doc_002", "INTERPRETS"),
 ("doc_001", "doc_004", "AMENDS"),
]

async def build_graph(graph: Neo4jGraph, documents: list[dict]) -> None:
 for doc in documents:
 await graph.upsert_node(doc["id"], {"title": doc["title"]})
 for src, tgt, rel in EDGES:
 await graph.upsert_edge(src, tgt, rel)

async def main():
 settings = QuantaSettings()
 graph = Neo4jGraph(
 uri=settings.NEO4J_URI,
 user=settings.NEO4J_USER,
 password=settings.NEO4J_PASSWORD,
)
 await build_graph(graph, my_documents)
 await graph.close()

asyncio.run(main())
```

See [examples/graph\\_search.py](#) for a complete runnable example with the legal documents use case.

## 9.6 Tuning `graph_seed_k` and `graph_hops`

`graph_seed_k` controls how many top-scoring dense hits are used as BFS seeds. `graph_hops` controls the traversal depth.



| Parameter                 | Too low                                  | Too high                                  |
|---------------------------|------------------------------------------|-------------------------------------------|
| <code>graph_seed_k</code> | Misses edges from lower-ranked documents | More noise; distant neighbours enter pool |
| <code>graph_hops</code>   | Misses multi-hop relationships           | Exponentially more candidates; slower     |

Recommended starting point: `graph_seed_k=3`, `graph_hops=2`. In practice, most useful graph paths are within 2 hops. A 3-hop expansion in a dense graph can add hundreds of candidates.

**Note:** `graph_weight` controls how much the graph-derived score ( $1 / \text{hop\_distance}$ ) contributes to the final ranking. Setting `graph_weight=0.0` turns the graph into a pure candidate expander: expanded documents enter the pool but do not receive any score boost. They will appear at the bottom of results unless they are also found by dense or BM25 search. Tune `graph_seed_k` and `graph_hops` to control how broadly the graph expands.

## 9.7 The source field

Every `RetrievalResult` carries a `source` string describing how the document was found:

| Value                      | Meaning                                                                     |
|----------------------------|-----------------------------------------------------------------------------|
| <code>"dense"</code>       | Found by vector search. May also have a BM25 score (not separately tagged). |
| <code>"graph"</code>       | Not found by vector search; added to the pool by graph expansion only.      |
| <code>"dense+graph"</code> | Found by vector search AND confirmed or boosted by graph expansion.         |

The current implementation tracks three signal paths. BM25 hits contribute to the score but are not reflected as a separate label in the source field.

```
for r in results:
 if r.source == "graph":
 print(f"{r.id} - reached only via graph traversal, score={r.score:.3f}")
 elif r.source == "dense+graph":
 print(f"{r.id} - found by search AND graph, score={r.score:.3f}")
 else:
 print(f"{r.id} - dense search result, score={r.score:.3f}")
```

## 9.8 Debugging graph retrieval

### Graph returns no results:

1. Check Neo4j connectivity: `await graph._driver.verify_connectivity()`
2. Confirm nodes exist: open Neo4j Browser (<http://localhost:7474>) and run `MATCH (d:Document) RETURN d LIMIT 10`
3. Confirm edges exist: `MATCH ()-[r]-() RETURN type(r), count(*)`
4. Check that node `id` properties match the IDs in your `QuantaIndex`

5. Check `graph_seed_k` — if it's 0, no seeds are passed to the graph

### Graph expansion returns seeds themselves:

The Neo4j `expand()` query uses `WHERE NOT neighbor.id IN $seeds`. Seeds are always excluded from graph results.

### Dense results look wrong:

Disable the graph (`use_graph=False`) to isolate whether the issue is in vector search or graph expansion.

### Enable debug logging:

```
import logging
logging.getLogger("quanta.retriever").setLevel(logging.DEBUG)
logging.getLogger("quanta.graph").setLevel(logging.DEBUG)
```

The retriever logs `dense_hits`, `bm25_hits`, `graph_hits`, and `returned` counts at `DEBUG` level on every search.

---

## 10. BM25 with Tantivy

### 10.1 When to use

Tantivy BM25 is most effective for corpora up to ~50,000 documents. Above that, index rebuild times and memory usage grow and the benefit over good dense search diminishes. Use it when:

- Queries contain rare or technical keywords that embedding models may conflate (e.g., product codes, legal article numbers, proper nouns).
- Your users submit keyword-style queries as well as natural language queries.
- You need hybrid precision: dense recall + keyword precision.

### 10.2 The add/commit workflow

`TantivyBM25` separates staging from committing. Staged documents are not searchable until `commit()` is called.

```
from quanta import TantivyBM25, QuantaSettings

settings = QuantaSettings()
bm25 = TantivyBM25(settings)

Stage documents
bm25.add("chunk-001", "The controller shall notify the supervisory authority.")
bm25.add("chunk-002", "Personal data must be processed lawfully.")

Commit to disk — now searchable
bm25.commit()
```

```
Search
results = bm25.search("controller notification", k=5)
for doc_id, score in results:
 print(doc_id, score)
```

**Bulk add:**

```
docs = [(f"chunk-{i}", text) for i, text in enumerate(my_texts)]
bm25.add_bulk(docs)
bm25.commit()
```

**Delete:**

```
bm25.delete("chunk-001")
bm25.commit()
```

### 10.3 Hybrid scoring

When `query_text` is provided to `retriever.search()` and a `TantivyBM25` instance is wired into `MultiRetriever`, all three weights are renormalised:

```
eff_dense = dense_weight / (dense_weight + bm25_weight + graph_weight)
eff_bm25 = bm25_weight / (dense_weight + bm25_weight + graph_weight)
eff_graph = graph_weight / (dense_weight + bm25_weight + graph_weight)
```

With defaults `dense_weight=0.5`, `bm25_weight=0.3`, `graph_weight=0.3`:

```
eff_dense = 0.5 / 1.1 ≈ 0.455
eff_bm25 = 0.3 / 1.1 ≈ 0.273
eff_graph = 0.3 / 1.1 ≈ 0.273
```

BM25 is only active when both `query_text` is passed to `search()` AND the `bm25` instance is not a `NullBM25`.

### 10.4 query\_text parameter

```
results = await retriever.search(
 query_vectors={"text": query_embedding},
 k=10,
 query_text="controller obligations GDPR", # enables BM25 leg
)
```

If `query_text` is `None`, BM25 is skipped even if `TantivyBM25` is configured.

---

## 11. Embedding Cache

### 11.1 Redis setup

```
docker compose --profile cache up -d
```

**.env:**

```
REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_PASSWORD= # leave blank if no auth
REDIS_DB=0
REDIS_TTL_SECONDS=86400 # 24 hours
```

```
from quanta import QuantaSettings, get_cache

settings = QuantaSettings()
cache = get_cache(settings) # RedisCache or NullCache
```

### 11.2 Usage

The cache is used via `MultiRetriever.get_or_embed()`:

```
embedding = retriever.get_or_embed(text, embed_fn=my_model.encode)
```

On a hit, `embed_fn` is never called. On a miss, the embedding is computed and stored with TTL `REDIS_TTL_SECONDS`.

### 11.3 Cache hit/miss logging

Set the `quanta.cache` logger to `DEBUG` to see hit/miss events. Misses log at `WARNING` level when Redis connection fails.

### 11.4 TTL strategy

| Use case                  | Recommended TTL      |
|---------------------------|----------------------|
| Static document corpus    | Long (days to weeks) |
| Frequently updated corpus | Short (hours)        |

| Use case                        | Recommended TTL          |
|---------------------------------|--------------------------|
| Query embeddings (user queries) | Short (minutes to hours) |
| Development / testing           | Any (or NullCache)       |

## 11.5 Graceful degradation

If Redis is unreachable at startup, `get_cache()` logs a warning and returns `NullCache`. If Redis becomes unreachable mid-operation, `RedisCache.get()` and `set()` catch all exceptions and return `None` / log a warning. The application continues without caching.

## 12. LlamaIndex Integration

### 12.1 QuantaVectorStore constructor

```
from quanta.integrations.llama_index import QuantaVectorStore

store = QuantaVectorStore(
 retriever: MultiRetriever, # must have index_name registered
 index_name: str, # which QuantaIndex to use for writes/reads
 embed_dim: int, # must match your embedding model's output
)
```

### 12.2 Write path: `async_add`

`async_add` takes a list of LlamaIndex `TextNode` objects. Each node must have:

- `node.node_id` — used as the chunk ID
- `node.embedding` — must be pre-computed (not `None`)
- `node.ref_doc_id` — used as the document ID (falls back to `node_id`)
- `node.metadata` — stored verbatim; `chunk_index` and `doc_type` keys are read if present

```
from llama_index.core.schema import TextNode
import numpy as np

node = TextNode(
 node_id="chunk-001",
 text="The controller shall implement appropriate measures.",
 embedding=my_model.encode("...").tolist(),
 metadata={"doc_type": "regulation", "chunk_index": 0},
)
added_ids = await store.async_add([node])
```

### 12.3 Delete path: `adelete`

```
await store.delete(ref_doc_id="parent-doc-id")
```

Deletes all chunks with `document_id == ref_doc_id` from both the docstore and the `QuantaIndex`.

## 12.4 Query path: aquery

```
from llama_index.core.vector_stores.types import VectorStoreQuery

result = await store.aquery(
 VectorStoreQuery(
 query_embedding=my_model.encode(query_text).tolist(),
 similarity_top_k=5,
)
)
for node_with_score in result.nodes:
 print(node_with_score.score, node_with_score.node.get_content()[:80])
```

## 12.5 Full VectorStoreIndex example

```
from llama_index.core import VectorStoreIndex, StorageContext

storage_ctx = StorageContext.from_defaults(vector_store=store)
li_index = VectorStoreIndex(nodes=[], storage_context=storage_ctx)

Use li_index.insert_nodes() or li_index.as_query_engine() as usual
The query engine calls aquery() internally
```

## 12.6 Supported operators

`aquery` passes `VectorStoreQuery.query_embedding` and `similarity_top_k` directly to `MultiRetriever.search()`. LlamaIndex `MetadataFilters` are not currently translated — pass `filters` at the `MultiRetriever` level if you need metadata pre-filtering.

# 13. Performance Tuning

## 13.1 bit\_width selection

| bit_width | Memory vs float32 | Recall impact    | Recommended for                        |
|-----------|-------------------|------------------|----------------------------------------|
| 1         | 32× smaller       | Significant loss | Extreme memory constraints only        |
| 2         | 16× smaller       | Moderate loss    | Large corpora where recall < 95% is ok |
| 4         | 8× smaller        | Small loss       | Most production use cases (default)    |
| 8         | 4× smaller        | Minimal loss     | When recall is critical                |

Start at 4. Move to 2 only if memory is a hard constraint and you have validated recall on your dataset.

## 13.2 graph\_seed\_k tuning

Start at 3–5. Increase if your graph has high branching and you see relevant graph-linked documents missing from results. The cost is  $O(\text{graph\_seed\_k} \times \text{graph\_hops})$  Neo4j queries per search.

## 13.3 DuckDB vs PostgreSQL

| Scenario                         | Recommended backend |
|----------------------------------|---------------------|
| Single-process indexing pipeline | DuckDB              |
| Local development                | DuckDB              |
| CI / unit tests                  | DuckDB (or mock)    |
| Production API with concurrency  | PostgreSQL          |
| Metadata GIN index needed        | PostgreSQL          |

DuckDB does not support concurrent writes from multiple processes. PostgreSQL scales with `POSTGRES_POOL_SIZE`.

## 13.4 Redis TTL impact

Large TTL values reduce cache misses but increase Redis memory usage. Monitor with `redis-cli INFO memory`. Eviction policy `allkeys-lru` is recommended for pure cache deployments.

## 13.5 Tantivy commit frequency

Every `commit()` triggers a Tantivy segment merge, which is I/O-bound. For batch indexing, call `commit()` once after all documents are added, not per document. For streaming ingestion, commit every 1,000–10,000 documents depending on your write latency tolerance.

# 14. Troubleshooting

| Problem                          | Likely cause                                                         | Solution                                                                                                     |
|----------------------------------|----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| Connection refused on PostgreSQL | Database not running or wrong host/port                              | <code>docker compose up -d postgres</code> ; check <code>POSTGRES_HOST</code> and <code>POSTGRES_PORT</code> |
| Connection refused on Neo4j      | Neo4j not running or not started with the <code>graph</code> profile | <code>docker compose --profile graph up -d</code> ; wait for <code>healthy</code> status                     |
| Connection refused on Redis      | Redis not running or not started with the <code>cache</code> profile | <code>docker compose --profile cache up -d</code>                                                            |

| Problem                                                                    | Likely cause                                                            | Solution                                                                                          |
|----------------------------------------------------------------------------|-------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| turbovec compilation error during install                                  | Missing Rust toolchain                                                  | Install Rust: <code>curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs   sh</code>         |
| tantivy not found or <code>ModuleNotFoundError: tantivy</code>             | BM25 extra not installed                                                | <code>pip install "quanta[bm25]"</code>                                                           |
| <code>QuantaError: vectors must have shape (n, dim)</code>                 | Wrong vector shape or wrong dim                                         | Check <code>idx._dim</code> ; ensure vectors are <code>(n, dim)</code> not <code>(dim,)</code>    |
| <code>QuantaError: len(ids)=... does not match vectors.shape[0]=...</code> | IDs list length $\neq$ number of vectors                                | Pass one ID per vector                                                                            |
| <code>QuantaError: xxhash-64 collision</code>                              | Two different string IDs hash to the same uint64                        | Extremely rare; rename one ID                                                                     |
| Graph returns no results                                                   | Nodes not inserted, IDs don't match, or <code>use_graph=False</code>    | Verify Neo4j has <code>:Document</code> nodes; confirm <code>id</code> property matches index IDs |
| Cache always misses                                                        | Redis unreachable or <code>REDIS_HOST</code> not set                    | Check <code>REDIS_HOST</code> in <code>.env</code> ; run <code>redis-cli ping</code>              |
| <code>QuantaError: Neo4j driver is required</code>                         | <code>neo4j</code> extra not installed                                  | <code>pip install "quanta[neo4j]"</code>                                                          |
| <code>DuckDBDocStore</code> is not initialised                             | <code>await docstore.init()</code> not called                           | Always call <code>await docstore.init()</code> before any operation                               |
| <code>PostgresDocStore</code> is not initialised                           | Same as above                                                           | Call <code>await docstore.init()</code> at startup                                                |
| Results have <code>content=None</code>                                     | Chunk not added to docstore                                             | Add the chunk with <code>add_chunk()</code> before or alongside <code>idx.add()</code>            |
| <code>QuantaError: index_name='...' not found in retriever</code>          | <code>QuantaVectorStore</code> <code>index_name</code> not in retriever | Pass <code>index_name</code> matching a key in <code>MultiRetriever.indexes</code>                |